

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Karish Kumar V	Reg. No.:	192524361
Section / Batch:	B	Date:	02/09/2026

Code of Conduct

I, Karish Kumar V 192524361 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Karish Kumar V

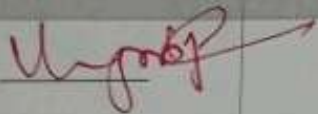
Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

convert the flowchart for breadth first search traversal into code.

```
#include <stdio.h>
int main()
{
    int a[10][10], q[10], v[10] = {0};
    int n, s, f = 0, r = 0, i, x;
    printf("Enter vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix: \n");
    for (i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    printf("Enter start vertex: ");
    scanf("%d", &s);
    q[r++] = s;
    v[s] = 1;
    printf("BFS: ");
    while (f < r) {
        x = q[f++];
        printf("%d", x);
        for (i = 0; i < n; i++)
            if (a[x][i] && !v[i]) {
                q[r++] = i;
                v[i] = 1;
            }
    }
    return 0;
}
```


2. Convert the flowchart for Kruskal's algorithm

```
#include <stdio.h>

int p[20];

int find(int x) {
    while (p[x] != x)
        x = p[x];
    return x;
}

int main() {
    int n, e, i, j, a, b, w, count = 0, cost = 0;
    int u[20], v[20], wt[20], t;
    scanf("%d %d", &n, &e);
    for (i = 0; i < e; i++)
        scanf("%d %d %d", &u[i], &v[i], &wt[i]);
    for (i = 0; i < n; i++)
        p[i] = i;
    for (i = 0; i < e + 1; i++)
        for (j = i + 1; j < e; j++)
            if (wt[i] > wt[j]) {
                t = wt[i]; wt[i] = wt[j]; wt[j] = t;
                t = u[i]; u[i] = u[j]; u[j] = t;
                t = v[i]; v[i] = v[j]; v[j] = t;
            }
    printf("MST: \n");
    for (i = 0; i < e & count < n - 1; i++) {
        a = find(u[i]);
        b = find(v[i]);
        printf("Minimum cost = %d", cost);
        return 0;
    }
}
```

convert the flowchart for topological sort into code.

```
#include <stdio.h>
int main() {
    int n, a[10][10], indeg[10] = {0};
    int q[10], front = 0, rear = 0;
    int i, j, v, count = 0;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if (a[i][j])
                indeg[j]++;
        }
    for (i = 0; i < n; i++)
        if (indeg[i] == 0)
            q[rear++] = i;
    printf("Topological order: ");
    while (front < rear) {
        v = q[front++];
        printf("%d ", v);
        count++;
    }
    if (count != n)
        printf("\n Cycle exist!");
    return 0;
}
```

4. Convert the flowchart for generating a DFS & Code.

```
#include <stdio.h>
int graph[10][10], visited[10], n;
void DFS(int v) {
    int i;
    visited[v] = 1;
    printf("%d ", v);
    for (i = 0; i < n; i++) {
        if (graph[v][i] && !visited[i]) {
            printf("-> ");
            DFS(i);
        }
    }
}

int main() {
    int i, j, start;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);
    scanf("%d", &start);
    printf("DFS Tree: ");
    DFS(start);
    return 0;
}
```


the flowchart for prim's algorithm into code.

```
#include <stdio.h>
```

```
#define INF 9999
```

```
int main() {
```

```
    int n, cost[10][10], visited[10] = {0};
```

```
    int i, j, edges = 0, min, u, v, total = 0;
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &cost[i][j]);
```

```
    visited[0] = 1;
```

```
    while (edges < n - 1) {
```

```
        min = INF;
```

```
        for (i = 0; i < n; i++)
```

```
            if (!visited[i])
```

```
                for (j = 0; j < n; j++)
```

```
                    printf("%d - %d = %d\n", u, v, min);
```

```
    visited[v] = 1;
```

```
    total += min;
```

```
    edges++;
```

```
}
```

```
    printf("minimum cost = %d", total);
```

```
    return 0;
```

Ujjwal